VANIER COLLEGE
TEACHER: SLEIMAN RABAH

420-411-VA ECOMMERCE – FALL 2025

COMPUTER SCIENCE DEPARTMENT

ASSIGNMENT #2

Due date: scheduled on LÉA

CONSTRAINTS

- This is an **individual** assignment,
- You are not permitted to collaborate nor share code with any of your classmates,
- Using ChatGPT, GitHub Copilot, or any similar tools is not permitted,
- It is your responsibility to be familiar with the Cheating and Plagiarism Policy (Section 11.4)
- **Read the above points five more times.**

LEARNING OBJECTIVES

By completing this assignment, you will be able to:

- Apply the MVC pattern to organize and structure a web application,
- Implement middleware for session management, authentication, and authorization in Slim,
- Develop a secure authentication and authorization system with role-based access,
- Perform CRUD operations for product and category management,
- Handle file uploads and validate uploaded content securely,
- Implement internationalization (i18n) using the Symfony Translation Component,
- Apply security best practices such as 2FA, password hashing, and input validation,
- Produce well-documented, maintainable code following PSR-12 standards.

REQUIRED TOOLS

- Wampoon and VS Code

OVERVIEW

In this assignment, you will design and implement a small yet functional eCommerce web application that simulates the core features of an online store.

The goal of this assignment is to apply professional web development practices, including clean code organization, authentication and authorization, middleware design, file handling, and adherence to security standards.

The project will be divided into **two main** components:

1) A frontend: A customer-facing section where customers can browse products, manage their shopping cart, and complete purchases; and

2) A backend (admin panel): Where store administrators can log in, manage the catalog, and handle store operations.

TEAM CONFIGURATION

You will work in teams of **two students**.

- Both teammates must share full responsibility for every aspect of the project, including database design, backend logic, and frontend functionality.
- Collaboration and **version control** (using Git and GitHub) are essential: make sure both of you contribute meaningfully and understand the entire system.

GETTING STARTED

Your website should use Bootstrap or Bluma for all the styling so that it looks clean and works well on both computers and phones. To avoid repeating code, you'll set up a few reusable components:

a) Create and use a private GitHub repository for this assignment.
b) Add the Slim-MVC starter template to your repo.
c) You must design the user interface yourself without using any pre-built or third-party HTML templates.
d) Database design: Please consult the webpage provided on the external course website.

IMPLEMENTATION REQUIREMENTS

A) ADMIN PANEL (BACKEND)

The admin panel is the management interface of your online store. It must be secure, accessible only to authorized administrators, and allow efficient management of products and categories.

1) Authentication and Authorization

Administrators must log in using valid credentials before accessing the admin panel. Once authenticated, they should be able to access administrative tools unavailable to regular users.

To strengthen security:

- Implement role-based access control to distinguish between administrators and regular users.
- Two-factor authentication (2FA): Add an extra security layer requiring a second verification step (like a code sent via email or authenticator app) when admins log in.

2) Product and Category Management

The main feature of the admin panel is the ability to perform CRUD (Create, Read, Update, Delete) operations on products and categories.

Administrators should be able to:

- View lists of all existing products and categories with relevant details.
- Create new products and categories, assigning each product to a category.
- Edit existing items to update their information, such as product prices or descriptions, etc.
- Delete products or categories that are no longer needed.
- Each product must belong to a category to ensure that the store's inventory remains organized.
- **Note:** Manage products and categories separately, each with its own controllers, models, and views.

3) File Upload

Each product should have at least one image, uploaded by the administrator. The file upload feature must:

- Allow uploading and associating images with products.
- Validate uploaded files by checking type and size.
- Implement security measures such as renaming files and storing them outside the public directory to prevent malicious uploads.
- Store the uploaded file path in the products table.

4) Order Management

Add a page in the admin panel where administrators can view a list of all customer orders stored in the database. This page should display a list of all ***customer orders*** stored in the database. Each order entry should include key details such as the Order ID, Customer Name or ID, Total Amount, Status, and Date Created.

B) FRONTEND (USER-FACING):

The frontend represents the customer-facing portion of your application, where users can browse, search, and purchase products.

1) Product Browsing

Customers should be able to view products organized by category, such as electronics, books, or clothing, etc.

- Each product listing should include its name, price, image, and description.
- To improve usability, implement an AJAX-based live search feature that dynamically filters products as the user types, without requiring a separate "Search" button.
- Products should be filtered by category.

2) User Authentication

Customers must be able to register for an account, log in securely, and log out when finished.
Your registration form should include validation to ensure data quality (for example, existing account, valid email format, required fields, and strong passwords).

Additionally:

- Implement session management to keep users logged in as they navigate.
- Include two-factor authentication (2FA) for users.
- Provide a reliable logout mechanism that ends the session securely.

3) Shopping Cart

A fully functional shopping cart is an essential feature. You must manage the shopping cart using PHP sessions.

Customers should be able to:

- Add products to their cart while browsing.
- View all items in their cart with product details, quantities, and prices.
- Update quantities directly in the cart.
- Remove unwanted items.
- See subtotal and total prices update automatically as they modify the cart.

4) Checkout Process

The checkout process should include:

- Proceed to Checkout: Provide a clear button or link from the cart page to initiate the checkout process.
- **Order summary:** Display a final review of items, quantities, prices, and the total cost before confirming the order.
- Order confirmation: After the user confirms the checkout, display a confirmation page indicating that the purchase was successful.
- Order storage: Once the checkout is confirmed, all order details, including the customer's ID, products purchased, quantities, prices, and total amount must be recorded in the orders table. Each product in the order should also be stored in the `order_items` table, maintaining a proper relationship between orders and items for future reference (e.g., order history, admin review).

C) Additional Features and Requirements:

5) Offline Two-Factor Authentication (2FA)

For this project, the two-factor authentication (2FA) mechanism should operate offline and use a QR code–based verification system. When an administrator or user enables 2FA, the system should generate a unique secret key that can be embedded in a QR code.

- The user scans this QR code with an authenticator app (such as Microsoft/Google Authenticator or Authy), which then generates time-based one-time passwords (TOTPs) on their device. During login, after entering the correct username and password, the user must provide the current verification code from their authenticator app. The system validates the code locally using the shared secret, without relying on an external server or internet connection. This ensures secure, reliable authentication even in environments with limited connectivity.
- Note: additional details and guidelines for implementing 2FA will be provided in class.

6) Flash Messages

Use session-based flash messages to give users (customers and admins) immediate feedback after performing actions. These short messages enhance user experience by confirming the outcome of their actions.

Flash messages can include:

- Success: For example, “Product added to cart successfully.”
- Error: For example, “Login failed. Incorrect password.”
- Warning: For example, “Your session is about to expire.”
- Information: For example, “Please verify your email address.”/ “Please scan the QR code.”
- Note: additional details and instructions will be provided in class.

7) Internationalization (i18n)

Your application must support at least two languages:

- Use the Symfony Translation Component to translate all text in your application.
- Support at least two languages (e.g., English and French).
- Language switcher: Provide a language switcher (such as a dropdown or flag icons) that allows users to change the site's language. When switched, all text content in the application should be updated accordingly.
- Note: additional details and instructions will be provided in class.

8) Service Layer for Input Validation

You are required to implement a **service component** between the controller and the model to handle user input validation.

- The service layer for input validation must be utilized in both the **frontend** (user-facing forms) and the **backend** (admin forms for managing categories and products). This ensures that all user inputs, whether coming from customer interactions or administrative actions, are consistently validated before being processed or stored in the database. By centralizing validation in the service layer, the application maintains data integrity and reduces duplication of validation logic across the system.
- Use the Result pattern to return validation outcomes, including success or failure information and any relevant error messages. This approach promotes separation of concerns, keeps controllers lightweight, and centralizes validation logic for maintainability and consistency.

Technical Implementation/Requirements:

Required Components:

- Responsive design and CSS framework Integration: Use Bootstrap or Bluma for all styling and responsive design. Your coffee shop website should look great on any device (Bootstrap helps with).
- SweetAlert2 or Bootstrap modals for displaying confirmation dialogs.
- **Use Bootstrap icons** or any **font icon set** for all interactive elements.
- Navigation Bar: Consistent navbar across all pages linking to main sections.
- Use the PHP Slim framework and the provided slim-mvc template
- Use JavaScript for interactive elements such as live search.

Security Requirements

- Use prepared statements with named placeholders for all database queries.
- Validate and sanitize all user inputs on both the client and server sides.
- Validate all user input on both the client and server sides.
- Password Hashing: Use bcrypt, never store plain-text passwords.
- Implement proper error handling with try-catch blocks.
- Never expose database errors directly to users.
- Use `htmlspecialchars()` when outputting any database values to prevent XSS attacks.

WHAT TO SUBMIT

Your final submission must include the following:

1. Source Code: A complete copy of your project, submitted as a ZIP archive.
2. Database Schema: A .sql file containing table creation scripts and some test data (INSERT statements).
3. Documentation:
   - A README.md file explaining how to install and run your application.
   - A Database Diagram (ERD) showing tables and their relationships. For this, you can use phpMyAdmin's Designer feature.
   - A User Guide describing how to use the admin and customer interfaces.
4. Demonstration: Be ready to present your application and demonstrate its features after submission.

EVALUATION RUBRIC:

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Needs Improvement (D/F)	Points
Functionality and compliance with the requirements	All pages work perfectly, all requirements met	Minor bugs, most requirements met	Some features missing or buggy	Major functionality issues	**/50**
Database Implementation	Perfect use of PDO, prepared statements, proper error handling	Good database practices with minor issues	Basic database implementation	Poor database practices or security issues	**/5**
Code Documentation	Comprehensive comments, clear function/class documentation	Good commenting throughout code	Some comments present	Minimal or no comments	**/5**

Coding Style & Structure	Consistent indentation, clean code, well-organized files	Good style with minor inconsistencies	Acceptable style, some organization issues	Poor formatting, disorganized code	**/5**
Naming Conventions	Clear, descriptive variable/function names throughout	Good naming with few exceptions	Adequate naming conventions	Unclear or inconsistent naming	**/5**
Security Implementation	Perfect input validation, sanitization, prepared statements	Good security practices with minor gaps	Basic security measures implemented	Poor or missing security practices	**/15**
User Interface & Design	Professional Bootstrap implementation, responsive design	Good UI with minor design issues	Basic Bootstrap usage, some responsiveness	Poor UI or non-responsive design	**/10**
PRG Pattern & Flash Messages	Perfect implementation of PRG pattern and session flash messages	Good implementation with minor issues	Basic implementation present	Missing or poorly implemented	**/5**
Total					**/100**